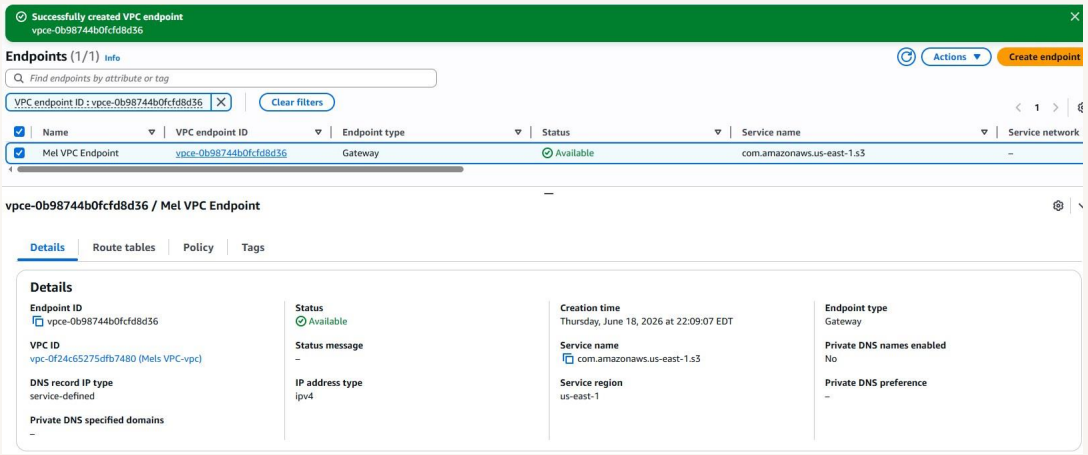
VPC Endpoints

Melvin green



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is a logically isolated section of the AWS cloud where you can launch resources like EC2 instances within a network that you define and control. It allows you to set your own IP address range, create subnets, configure route tables, and control inbound/outbound traffic using security groups and network ACLs. VPCs are useful because they give you full control over your network environment, similar to how you'd

manage a traditional on-premises data center, but with the scalability and flexibility of the cloud.

How I used Amazon VPC in this project

In this project, I used Amazon VPC as the foundation for the entire architecture. I started by launching a VPC (using the VPC wizard), which automatically created a public subnet, an internet gateway, and a route table. Within this VPC, I launched an EC2 instance and configured a security group to control its traffic. To enable private connectivity between the EC2 instance and my S3 bucket, I

created a VPC Gateway Endpoint. This added a route to the VPC's route table that directed S3-bound traffic straight to the endpoint instead of out through the internet gateway. I then layered on a bucket policy restricting S3 access to only that VPC endpoint, and an endpoint policy to control what the endpoint itself was allowed to do. Together, these pieces demonstrated how a VPC can be used not just to host compute resources, but to architect secure, private pathways to other AWS services, removing reliance on the public internet entirely.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was that creating the VPC endpoint wasn't enough on its own to enable private connectivity. Even after setting it up, my EC2 instance was still trying to reach S3 over the internet, and I had to manually update the route table to point S3-bound traffic to the endpoint.

In the first part of my project...

Step 1 - Architecture set up

In this step, I create a VPC to host an EC2 instance, establishing the network environment in which the instance will run. I then create an S3 bucket, which will later be accessed privately from the EC2 instance via a VPC endpoint.

Step 2 - Connect to EC2 instance

In this step, I connect to my EC2 instance to begin testing access to the S3 bucket from within the VPC.

Step 3 - Set up access keys

In this step, I create an access key to grant my EC2 instance access to my S3 bucket.

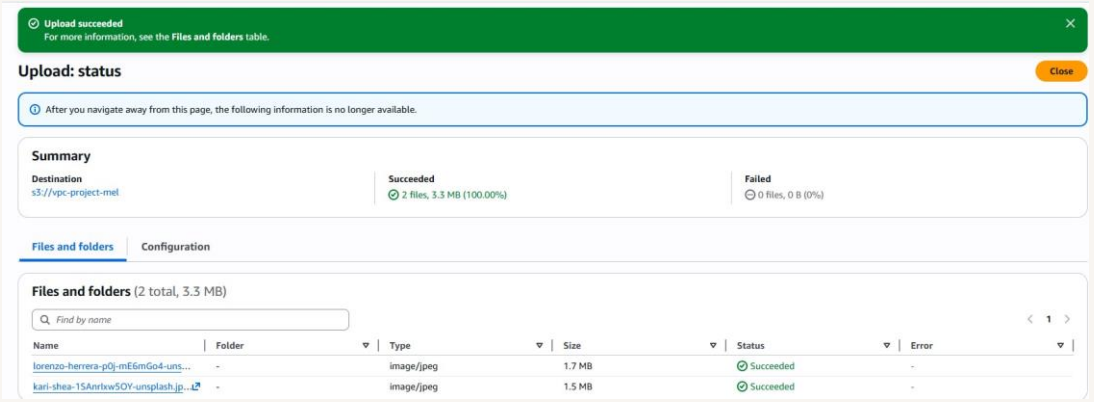
Step 4 - Interact with S3 bucket

In this step, I return to my EC2 instance and use my newly created access key to access my S3 bucket. Now that the access key is configured, I should be able to successfully run S3 commands from the instance.

Architecture set up

I started this project by launching my VPC using the VPC resource wizard, which automatically created my VPC, public subnet, internet gateway, and route table. Once that was complete, I launched an EC2 instance within the VPC and subnet, while also configuring a security group to control inbound and outbound traffic to the instance.

I also created my S3 bucket and uploaded two files to it.



Access keys

Credentials

To set up my EC2 instance to interact with my AWS environment, I ran aws configure and entered my access key ID and secret access key. I then configured my default region; it's important that this matches the region where my S3 bucket and VPC are located, since cross-region access can cause connectivity issues. Finally, I set the default output format, which can be JSON, text, table, or YAML.

Access keys are credentials made up of an access key ID and a secret access key, which are used to grant a user (or application) programmatic access to specific AWS resources.

A secret access key functions like the password that pairs with my access key ID. Together, these two credentials are required to authenticate and access AWS services programmatically.

Best practice

Although I'm using access keys in this project, a best practice alternative is to create an IAM role and attach it to the instance instead. I chose to use an access key here for the purposes of this demonstration.

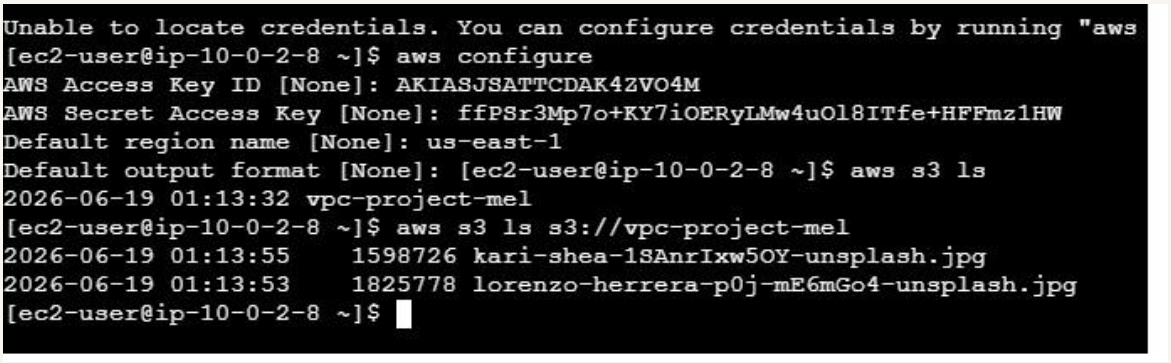
Connecting to my S3 bucket

The command I ran was aws s3 ls. This command lists the S3 buckets accessible from my AWS account, which I used to verify that my EC2 instance could reach my S3 bucket over the network.

The terminal responded by listing my S3 bucket, vpc-project-mel, confirming that the access keys were configured correctly and that I could now access my S3 bucket from my EC2 instance.

Connecting to my S3 bucket

I also tested the command aws s3 ls s3://vpc-project-mel, which returned the objects uploaded to my bucket, showing their file names.

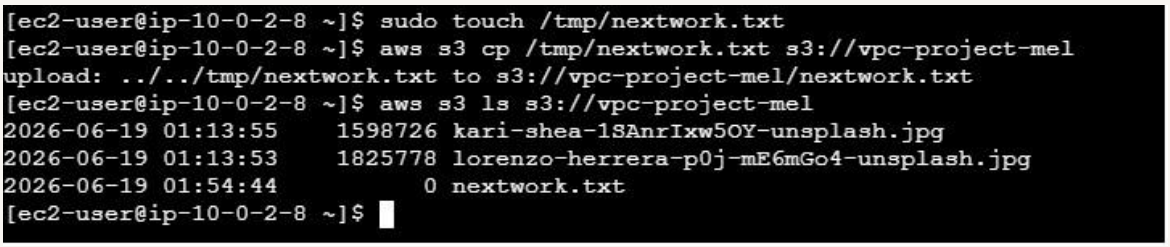


Uploading objects to S3

To prepare a file to upload to my bucket, I ran sudo touch /tmp/nextwork.txt. This command created a blank .txt file named nextwork.txt in the /tmp directory on my EC2 instance.

The second command I ran was aws s3 cp /tmp/nextwork.txt s3://vpc-projectmel. This command uploads the nextwork.txt file from my EC2 instance to my S3 bucket.

The third command I ran was aws s3 ls s3://vpc-project-mel, which validated that my file was uploaded successfully by listing the contents of the bucket.



In the second part of my project...

Step 5 - Set up a Gateway

In this step, I create a VPC endpoint to allow my EC2 instance to communicate directly with my S3 bucket, instead of routing traffic over the public internet.

Step 6 - Bucket policies

In this step, I limit access to my S3 bucket so that it only accepts traffic from the VPC endpoint. This will demonstrate that my S3 bucket and EC2 instance are communicating through the private route created by the gateway endpoint, rather than over the public internet.

Step 7 - Update route tables

In this step, I attempt to access my S3 bucket from my EC2 instance. If this succeeds, it will confirm that the VPC endpoint is correctly routing traffic and that the bucket policy is working as intended. If it fails, I'll need to troubleshoot potential connectivity issues.

Step 8 - Validate endpoint conection

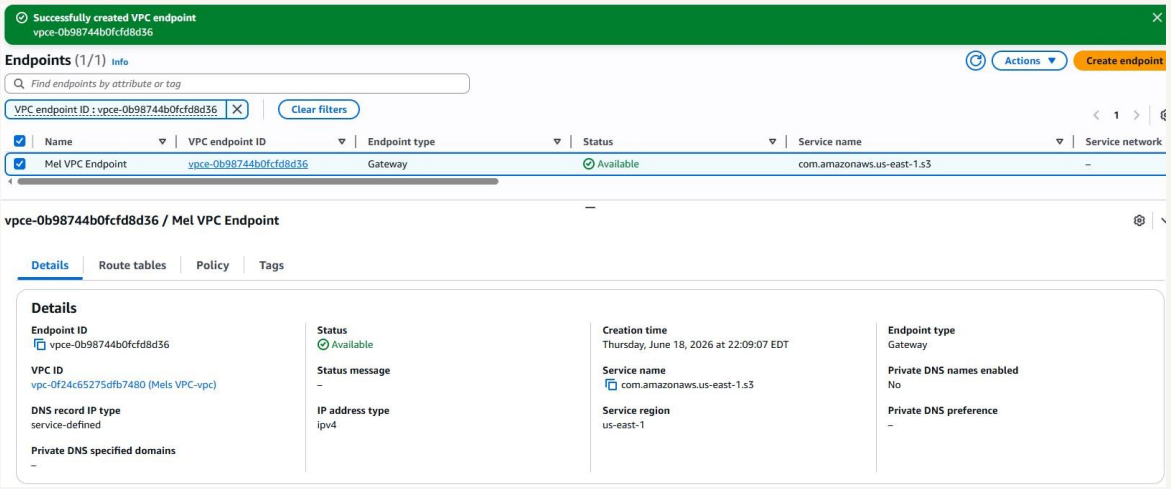
After adding the VPC endpoint to the route table, I will retest access to my S3 bucket from my EC2 instance.

Setting up a Gateway

I set up an S3 Gateway endpoint, which is a type of VPC endpoint. A gateway endpoint works by adding a route to the VPC's route table that directs traffic bound for S3 to go straight to the gateway instead of out to the internet.

What are endpoints?

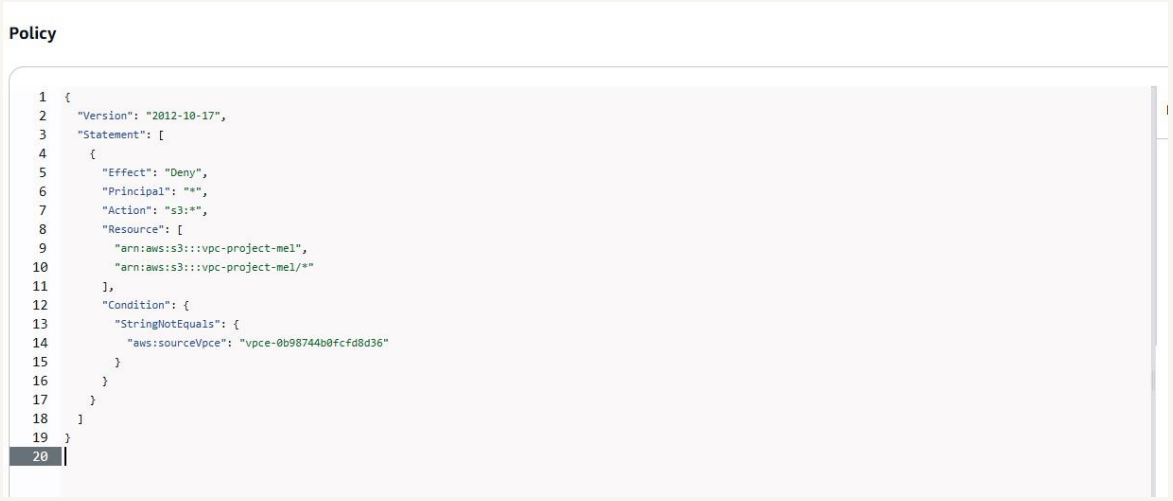
An endpoint is a private connection that allows my VPC to communicate with other AWS services securely, without traversing the public internet. For instance, S3 exists outside of any VPC because it's designed as a regional service with high availability and durability, accessible from anywhere. A VPC endpoint, then, creates a private connection between my VPC and an AWS service like S3 — in this case, an S3 endpoint.



Bucket policies

A bucket policy is a type of resource-based policy used to control access to my S3 bucket. By using a bucket policy, I get to decide who has access to the bucket and what actions they're allowed to perform.

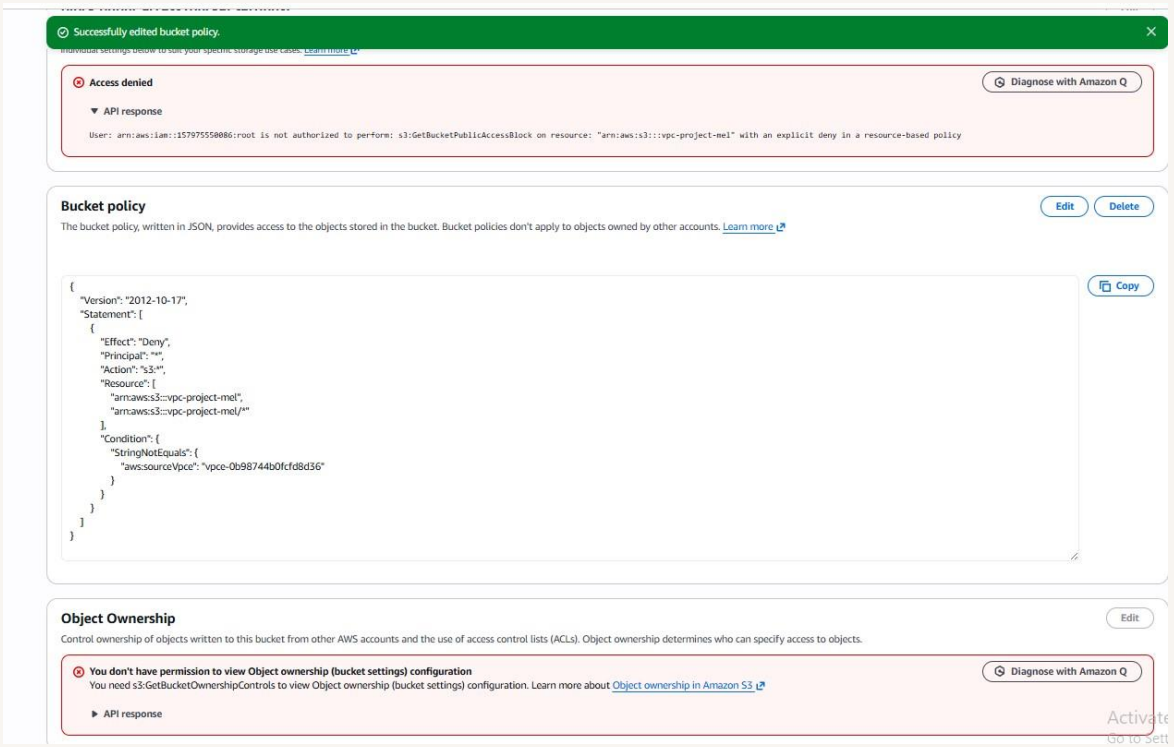
My bucket policy denies all actions on my S3 bucket and its objects to everyone, unless the request comes from the VPC endpoint with the ID defined in the aws:sourceVpce condition. This effectively grants access exclusively to my VPC endpoint, blocking all other traffic.



Bucket policies

Right after saving my bucket policy, my S3 bucket page showed "Access Denied" warnings. This is because the policy denies all actions unless the request comes from the VPC endpoint, meaning even accessing the S3 bucket through the AWS Management Console is now blocked.

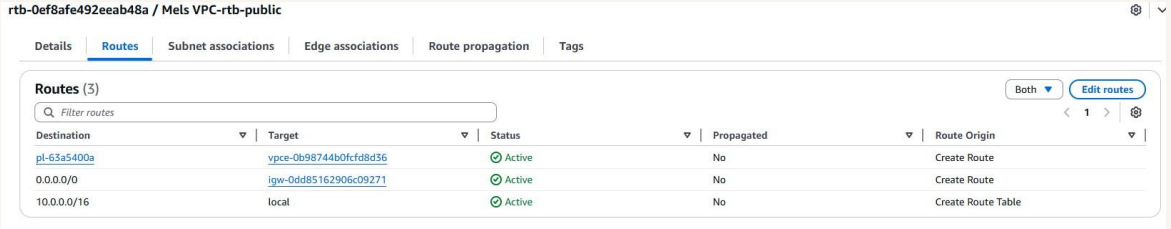
I also had to update my route table because my EC2 instance was still attempting to reach my S3 bucket over the internet instead of through the VPC endpoint. Since my bucket policy denies all traffic except requests from the VPC endpoint, the internet-routed traffic was being blocked.



Route table updates

To update my route table, I added my VPC endpoint as a route. I navigated to the VPC endpoint's settings, selected the Route Tables tab, and associated it with my VPC's route table.

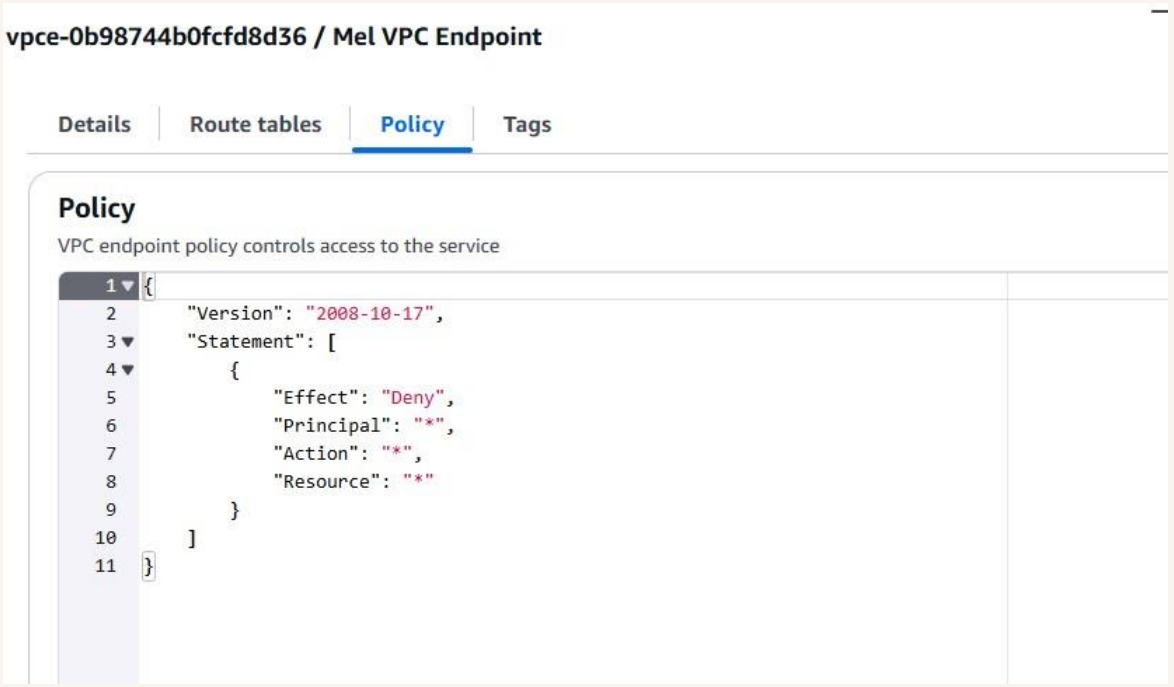
After updating my public subnet's route table, my terminal successfully returned the objects listed in my S3 bucket, confirming that my EC2 instance is now connecting to my S3 bucket over a private network via the VPC endpoint, rather than over the public internet as it had been doing before.



Endpoint policies

An endpoint policy is a policy attached directly to the VPC endpoint that controls what traffic is allowed to pass through it. This lets me get more granular about which resources can be accessed through the endpoint, for example, restricting access to specific S3 buckets or actions, rather than allowing the endpoint to reach any S3 resource. It could also serve as a quick way to lock down access if I suspected the endpoint was being misused or compromised.

I updated my endpoint's policy by switching the Effect in the policy document from Allow to Deny. I saw the effect of this change immediately — when I went back to my EC2 instance and tried to access my S3 bucket, I got an Access Denied error because I was no longer authorized to access the resource.


vpce-0b98744b0fcfd8d36 / Mel VPC Endpoint
Details
Route tables
Policy
Tags
Policy
VPC endpoint policy controls access to the service
{
"Version": "2008-10-17",
"Statement": [
{
"Effect": "Deny",
"Principal": "*",
"Action": "*",
"Resource": "*"
}
]
}